

Introduction

Behavioral design patterns are focused on the interaction and delegation of responsibilities between objects. One such pattern is the **Chain of Responsibility Pattern**, which allows a request to be passed along a chain of potential handlers until one of them processes it. This pattern is particularly useful for scenarios where multiple handlers might be responsible for processing a request, and we want to avoid tightly coupling the sender of the request to the receivers.

Imagine you're building a customer support system with different levels of support, such as basic inquiries, technical issues, and advanced troubleshooting. A customer can send in a request, and depending on its complexity, the request should be forwarded to the appropriate team. Instead of each team individually checking if they can handle every possible request, the **Chain of Responsibility Pattern** sets up a chain where each team can either process the request or pass it to the next team in the chain. This enables a flexible and extensible system where adding new handlers (teams) is easy and doesn't require changes to the existing code.

In the following sections, we will explore the **Chain of Responsibility Pattern** in more detail, understanding how it helps in decoupling sender and receiver objects, and how it creates a flexible chain of handlers for processing requests in various contexts.

Chain of Responsibility

Formal Definition

The **Chain of Responsibility Pattern** is a behavioral design pattern that transforms particular behaviors into standalone objects called handlers. It

allows a request to be passed along a chain of handlers, where each handler decides whether to process the request or pass it to the next handler in the chain.

This pattern decouples the sender of a request from its receivers, giving multiple objects a chance to handle the object.

Key Components

This pattern consists of the following components:

- **Handler:** An abstract class or interface that defines the method for handling requests and a reference to the next handler in the chain.
- **Concrete Handler:** A class that implements the handler and processes the request if it can. Otherwise, it forwards the request to the next handler.
- **Client:** The object that sends the request, typically unaware of the specific handler that will process it.

Real-Life Analogy: Customer Support

Imagine you're working in a customer support system. A customer submits a request that can be handled by multiple support teams, such as basic inquiries, technical issues, or billing problems. The **Chain of Responsibility Pattern** allows the system to forward the request through a chain of support teams (handlers), with each team deciding if they can resolve the issue or pass it along to the next team. This ensures that each team handles only the requests they are best suited to process, and the customer remains unaware of the chain of responsibility.

How It Works

The client sends a request to the first handler in the chain. If that handler can process the request, it does so. If not, it forwards the request to the next handler. This continues until either the request is handled or the end

of the chain is reached. The pattern allows for flexibility by enabling new handlers to be added to the chain without altering existing code. Let's now understand the working of the **Chain of Responsibility Pattern** through a problem statement.

Understanding the Problem

Assume we are building a customer support system for an e-commerce platform, where users raise tickets that can be of various types, such as general inquiries, refund requests, technical issues, and delivery complaints.

The following code snippet demonstrates how this functionality might be implemented in a naive way:

```
#include <iostream>
```

```
#include <string>
```

```
using namespace std;
```

```
// SupportService class: Handles different types of support requests
```

```
class SupportService {
```

```
    // Method to handle the support request based on the type of issue
```

```
public:
```

```
    void handleRequest(string type) {
```

```
        if (type == "general") {
```

```
        cout << "Handled by General Support" << endl;
    } else if (type == "refund") {
        cout << "Handled by Billing Team" << endl;
    } else if (type == "technical") {
        cout << "Handled by Technical Support" << endl;
    } else if (type == "delivery") {
        cout << "Handled by Delivery Team" << endl;
    } else {
        cout << "No handler available" << endl;
    }
}
};
```

// Main function: Entry point to test the chain of responsibility pattern

```
int main() {
    // Create an instance of SupportService
    SupportService supportService;

    // Test with different types of requests
    supportService.handleRequest("general");
    supportService.handleRequest("refund");
}
```

```
supportService.handleRequest("technical");

supportService.handleRequest("delivery");

supportService.handleRequest("unknown");


return 0;

}


import java.util.*;


// SupportService class: Handles different types of support requests
class SupportService {


// Method to handle the support request based on the type of issue
public void handleRequest(String type) {

    if (type.equals("general")) {

        System.out.println("Handled by General Support");

    } else if (type.equals("refund")) {

        System.out.println("Handled by Billing Team");

    } else if (type.equals("technical")) {

        System.out.println("Handled by Technical Support");

    } else if (type.equals("delivery")) {
```

```
        System.out.println("Handled by Delivery Team");
    } else {
        System.out.println("No handler available");
    }
}
}

// Main class: Entry point to test the chain of responsibility pattern
public class Main {

    public static void main(String[] args) {
        // Create an instance of SupportService
        SupportService supportService = new SupportService();

        // Test with different types of requests
        supportService.handleRequest("general");
        supportService.handleRequest("refund");
        supportService.handleRequest("technical");
        supportService.handleRequest("delivery");
        supportService.handleRequest("unknown");
    }
}
```

```
}
```

```
# SupportService class: Handles different types of support requests
```

```
class SupportService:
```

```
    # Method to handle the support request based on the type of issue
```

```
    def handle_request(self, type):
```

```
        if type == "general":
```

```
            print("Handled by General Support")
```

```
        elif type == "refund":
```

```
            print("Handled by Billing Team")
```

```
        elif type == "technical":
```

```
            print("Handled by Technical Support")
```

```
        elif type == "delivery":
```

```
            print("Handled by Delivery Team")
```

```
        else:
```

```
            print("No handler available")
```

```
# Main function: Entry point to test the chain of responsibility pattern
```

```
if __name__ == "__main__":
```

```
    # Create an instance of SupportService
```

```
support_service = SupportService()
```

```
# Test with different types of requests
```

```
support_service.handle_request("general")
```

```
support_service.handle_request("refund")
```

```
support_service.handle_request("technical")
```

```
support_service.handle_request("delivery")
```

```
support_service.handle_request("unknown")
```

Issues in the Above Code

Issue	Description
Violation of the Open-Closed Principle	Every time a new type of request is added, the <code>handleRequest</code> method must be modified, violating the Open-Closed Principle , which states that a class should be open for extension but closed for modification.
Monolithic Code	All logic is contained within a single method, making it difficult to maintain, test, and extend the system. Each handler (support team) is tightly coupled with the others.

Scalability and Flexibility	As the number of support teams grows, we cannot change the order of processing without modifying the core logic. It makes adding new handlers or changing the order of requests cumbersome.
------------------------------------	---

The Solution

The previous code implementation can be refactored using the Chain of Responsibility principle, which helps break down the system into individual handlers responsible for different types of requests. Each handler will either process the request or pass it on to the next handler in the chain, creating a more modular and maintainable structure.

In this refactor, the **SupportHandler** class acts as the base class, and each specific support type (**General**, **Billing**, **Technical**, **Delivery**) will extend the **SupportHandler** class. This allows us to create a chain of responsibility, where each handler checks if it can process the request and, if not, passes it to the next handler in the chain.

Code

```
#include <iostream>

#include <string>

using namespace std;

// Abstract class defining the SupportHandler
class SupportHandler {
```

protected:

```
    SupportHandler* nextHandler;
```

public:

```
    // Method to set the next handler in the chain
```

```
    void setNextHandler(SupportHandler* nextHandler) {  
        this->nextHandler = nextHandler;  
    }
```

```
    // Pure virtual method to handle the request
```

```
    virtual void handleRequest(string requestType) = 0;  
};
```

```
// Concrete Handler for General Support
```

```
class GeneralSupport : public SupportHandler {
```

public:

```
    void handleRequest(string requestType) override {
```

```
        if (requestType == "general") {
```

```
            cout << "GeneralSupport: Handling general query" << endl;
```

```
        } else if (nextHandler != nullptr) {
```

```
            nextHandler->handleRequest(requestType);
```

```
    }  
}  
};
```

```
// Concrete Handler for Billing Support
```

```
class BillingSupport : public SupportHandler {  
  
public:  
  
    void handleRequest(string requestType) override {  
  
        if (requestType == "refund") {  
  
            cout << "BillingSupport: Handling refund request" << endl;  
  
        } else if (nextHandler != nullptr) {  
  
            nextHandler->handleRequest(requestType);  
  
        }  
  
    }  
  
};
```

```
// Concrete Handler for Technical Support
```

```
class TechnicalSupport : public SupportHandler {  
  
public:  
  
    void handleRequest(string requestType) override {  
  
        if (requestType == "technical") {
```

```

        cout << "TechnicalSupport: Handling technical issue" << endl;
    } else if (nextHandler != nullptr) {
        nextHandler->handleRequest(requestType);
    }
}
};

```

// Concrete Handler for Delivery Support

```

class DeliverySupport : public SupportHandler {
public:
    void handleRequest(string requestType) override {
        if (requestType == "delivery") {
            cout << "DeliverySupport: Handling delivery issue" << endl;
        } else if (nextHandler != nullptr) {
            nextHandler->handleRequest(requestType);
        } else {
            cout << "DeliverySupport: No handler found for request" << endl;
        }
    }
};

```

```
// Client Code
```

```
int main() {
```

```
    SupportHandler* general = new GeneralSupport();
```

```
    SupportHandler* billing = new BillingSupport();
```

```
    SupportHandler* technical = new TechnicalSupport();
```

```
    SupportHandler* delivery = new DeliverySupport();
```

```
    // Setting up the chain: general -> billing -> technical -> delivery
```

```
    general->setNextHandler(billing);
```

```
    billing->setNextHandler(technical);
```

```
    technical->setNextHandler(delivery);
```

```
    // Testing the chain of responsibility with different request types
```

```
    general->handleRequest("refund");
```

```
    general->handleRequest("delivery");
```

```
    general->handleRequest("unknown");
```

```
    // Cleaning up memory
```

```
    delete general;
```

```
    delete billing;
```

```
    delete technical;
```

```
delete delivery;
```

```
return 0;
```

```
}
```

```
import java.util.*;
```

```
// Abstract class defining the SupportHandler
```

```
abstract class SupportHandler {
```

```
    protected SupportHandler nextHandler;
```

```
// Method to set the next handler in the chain
```

```
public void setNextHandler(SupportHandler nextHandler) {
```

```
    this.nextHandler = nextHandler;
```

```
}
```

```
// Abstract method to handle the request
```

```
public abstract void handleRequest(String requestType);
```

```
}
```

```
// Concrete Handler for General Support
```

```
class GeneralSupport extends SupportHandler {  
    public void handleRequest(String requestType) {  
        if (requestType.equalsIgnoreCase("general")) {  
            System.out.println("GeneralSupport: Handling general query");  
        } else if (nextHandler != null) {  
            nextHandler.handleRequest(requestType);  
        }  
    }  
}
```

// Concrete Handler for Billing Support

```
class BillingSupport extends SupportHandler {  
    public void handleRequest(String requestType) {  
        if (requestType.equalsIgnoreCase("refund")) {  
            System.out.println("BillingSupport: Handling refund request");  
        } else if (nextHandler != null) {  
            nextHandler.handleRequest(requestType);  
        }  
    }  
}
```

// Concrete Handler for Technical Support

```
class TechnicalSupport extends SupportHandler {  
  
    public void handleRequest(String requestType) {  
  
        if (requestType.equalsIgnoreCase("technical")) {  
  
            System.out.println("TechnicalSupport: Handling technical issue");  
  
        } else if (nextHandler != null) {  
  
            nextHandler.handleRequest(requestType);  
  
        }  
  
    }  
  
}
```

// Concrete Handler for Delivery Support

```
class DeliverySupport extends SupportHandler {  
  
    public void handleRequest(String requestType) {  
  
        if (requestType.equalsIgnoreCase("delivery")) {  
  
            System.out.println("DeliverySupport: Handling delivery issue");  
  
        } else if (nextHandler != null) {  
  
            nextHandler.handleRequest(requestType);  
  
        } else {  
  
            System.out.println("DeliverySupport: No handler found for request");  
  
        }  
  
    }  
  
}
```



```
}  
  
}  
  
// Client Code  
  
class Main {  
  
    public static void main(String[] args) {  
  
        SupportHandler general = new GeneralSupport();  
  
        SupportHandler billing = new BillingSupport();  
  
        SupportHandler technical = new TechnicalSupport();  
  
        SupportHandler delivery = new DeliverySupport();  
  
  
        // Setting up the chain: general -> billing -> technical -> delivery  
  
        general.setNextHandler(billing);  
  
        billing.setNextHandler(technical);  
  
        technical.setNextHandler(delivery);  
  
  
        // Testing the chain of responsibility with different request types  
  
        general.handleRequest("refund");  
  
        general.handleRequest("delivery");  
  
        general.handleRequest("unknown");  
  
    }  
}
```

```
}
```

```
# Abstract class defining the SupportHandler
```

```
class SupportHandler:
```

```
    def __init__(self):
```

```
        self.next_handler = None
```

```
# Method to set the next handler in the chain
```

```
def set_next_handler(self, next_handler):
```

```
    self.next_handler = next_handler
```

```
# Abstract method to handle the request
```

```
def handle_request(self, request_type):
```

```
    raise NotImplementedError
```

```
# Concrete Handler for General Support
```

```
class GeneralSupport(SupportHandler):
```

```
    def handle_request(self, request_type):
```

```
        if request_type.lower() == "general":
```

```
            print("GeneralSupport: Handling general query")
```

```
elif self.next_handler:
```

```
    self.next_handler.handle_request(request_type)
```

```
# Concrete Handler for Billing Support
```

```
class BillingSupport(SupportHandler):
```

```
    def handle_request(self, request_type):
```

```
        if request_type.lower() == "refund":
```

```
            print("BillingSupport: Handling refund request")
```

```
        elif self.next_handler:
```

```
            self.next_handler.handle_request(request_type)
```

```
# Concrete Handler for Technical Support
```

```
class TechnicalSupport(SupportHandler):
```

```
    def handle_request(self, request_type):
```

```
        if request_type.lower() == "technical":
```

```
            print("TechnicalSupport: Handling technical issue")
```

```
        elif self.next_handler:
```

```
            self.next_handler.handle_request(request_type)
```

Concrete Handler for Delivery Support

class DeliverySupport(SupportHandler):

def handle_request(self, request_type):

if request_type.lower() == "delivery":

print("DeliverySupport: Handling delivery issue")

elif self.next_handler:

self.next_handler.handle_request(request_type)

else:

print("DeliverySupport: No handler found for request")

Client Code

if __name__ == "__main__":

general = GeneralSupport()

billing = BillingSupport()

technical = TechnicalSupport()

delivery = DeliverySupport()

Setting up the chain: general -> billing -> technical -> delivery

general.set_next_handler(billing)

```
billing.set_next_handler(technical)
```

```
technical.set_next_handler(delivery)
```

```
# Testing the chain of responsibility with different request types
```

```
general.handle_request("refund")
```

```
general.handle_request("delivery")
```

```
general.handle_request("unknown")
```

How Chain of Responsibility Fixes the Previously Discussed Issues

Issue	Solution in Refactored Code
Violation of the Open-Closed Principle	Now, new types of requests can be handled by adding a new handler class without modifying the existing code. Each handler is open for extension and closed for modification.
Monolithic Code	The logic is now separated into individual handler classes, each responsible for one type of request, making the code more modular and easier to maintain.
Scalability and Flexibility	The chain of responsibility allows new handlers to be easily added without changing the existing logic. The

	order of handling can be changed by simply rearranging the chain.
--	---

When to Use

The Chain of Responsibility pattern is useful in several scenarios, particularly when you need to manage complex systems that involve multiple objects or entities interacting with each other. Here are some common situations where this pattern should be considered:

- **When multiple objects can handle a request, but the handler is not known beforehand:**

This pattern is ideal when you have several objects capable of processing a request, but you don't know which one should handle it at the time the request is made. The request is passed along the chain of handlers until one of them is able to process it.

- **When you want to decompose requests, senders, and receivers:**

In systems where requests can be processed by different entities, this pattern allows you to decouple the sender of the request from the receivers, making it easier to manage the flow of requests across various objects.

- **When you want to dynamically specify the chain of processing:**

The Chain of Responsibility allows you to dynamically alter the sequence of handlers that process a request. You can adjust the chain based on various conditions, offering greater flexibility in how requests are handled.

This pattern is particularly valuable in situations that require flexibility and scalability, such as customer support systems, event handling systems, or any application where a sequence of actions or checks need to be performed based on varying conditions.

Advantages and Disadvantages of Chain of Responsibility

Like any design pattern, the **Chain of Responsibility** pattern comes with its advantages and challenges. Below, we explore both the pros and cons of using this pattern in your systems.

Pros

- **Reduces coupling between sender and receiver:**
The sender of a request does not need to know which handler will process it. This decoupling allows for more flexibility in the system and reduces dependencies between objects.
- **Easy to add or remove handlers without changing existing code:**
New handlers can be introduced into the chain or existing handlers can be removed without modifying the existing code structure, making the system more maintainable.
- **Follows Single Responsibility Principle (SRP) and Open-Closed Principle (OCP):**
Each handler has a single responsibility (processing a specific type of request), and the system is open for extension (by adding new handlers) but closed for modification (no need to alter existing handlers).
- **Dynamic control over handler execution sequence:**
The chain of handlers can be dynamically configured, allowing you to adjust the order in which handlers process requests.

Cons

- **May lead to performance issues if the chain is too long:**
If there are too many handlers in the chain, the request may need to

pass through many objects before being processed, potentially leading to performance problems.

- **Debugging can be harder due to the dynamic flow of the chain:**

The dynamic nature of the chain can make it difficult to trace the flow of execution, making debugging more complex.

- **Risk of request not being handled at all:**

If none of the handlers in the chain can process the request, it may end up unhandled, which could lead to issues in the system if no fallback mechanism is implemented.

- **Sequence is important, as the order can break the logic:**

The order in which handlers are set up in the chain is crucial. If the order is incorrect, the request may be passed along the wrong sequence of handlers, breaking the logic of processing.

Real-Life Examples

There are various real-world applications of **Chain of Responsibility** principle. Some of them are:

1. Sign-Up Process in Website

In many online platforms, the sign-up process involves multiple steps, such as validating the email, confirming user age, checking for terms and conditions acceptance, and verifying CAPTCHA. Each of these checks can be handled by separate handlers (or objects), with each handler either processing the request or passing it to the next handler in the sequence. The chain of responsibility allows these checks to be dynamically handled in a sequence without tightly coupling them together.

2. Customer Support Ticket Routing

A typical customer support system can involve multiple departments such as general inquiries, billing, technical issues, and delivery problems. When a user submits a request, it needs to be passed through a chain of handlers, where each handler checks if the request matches its domain. For instance, the billing department handles refund requests, the technical support team handles technical issues, and so on. This approach helps keep the system flexible and extensible without changing the core logic every time a new department is added.

3. Event Handling in GUI Applications

In graphical user interface (GUI) applications, events like mouse clicks, keyboard presses, or other actions may be handled by multiple listeners. For example, a button click might trigger a series of events that are passed along a chain of responsibility. Each handler (listener) checks if it should process the event (like updating the button's state) or pass it to the next listener in the chain. This approach makes it easy to manage complex event-driven systems while keeping the components loosely coupled.

Class Diagram

